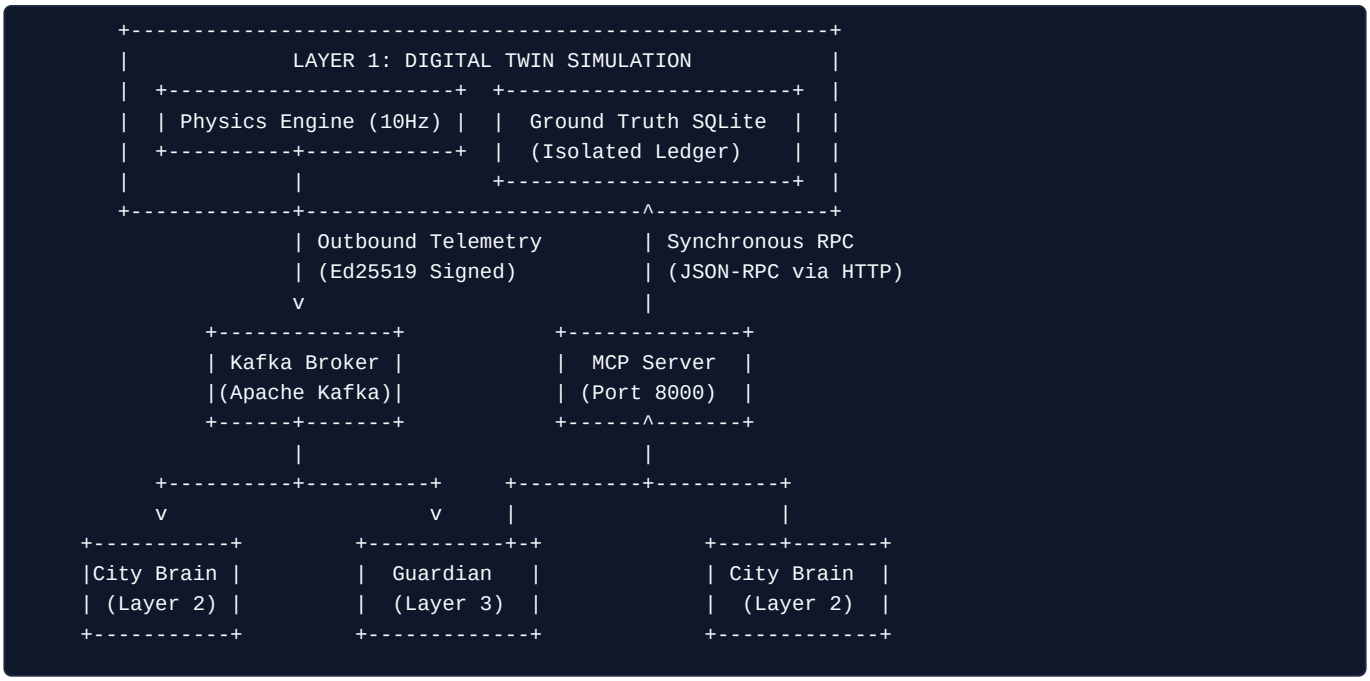
Twin Layer: Developer & Integration Guide

This guide walks you through setting up, running, testing, and integrating with Layer 1 (Twin Layer). Whether you are developing Layer 2 (City Brain), Layer 3 (Guardian), or contributing to Layer 1, this document provides the exact commands, code snippets, and network parameters you need.

---

1. System Overview & Architecture

Layer 1 simulates 42 physical IoT devices across 6 geographic zones in Greater Tunis. It runs a deterministic 10 Hz simulation loop (100 ms fixed timestep) driven by diurnal mathematical curves (traffic cycles, water drainage kinetics, electrical loads, particulate diffusion, and Poisson emergency call distributions).



Core Architecture Rules

1. **Unidirectional Streaming on Kafka:** Twin Layer streams telemetry to the Kafka topic `twin.sensors`. Partners only consume from this topic; never publish data to it.
2. **Synchronous Commands over MCP:** Actuation commands (`dispatch` , `set_valve` , etc.) and queries (`get_device_key` , `list_devices`) run strictly over the Model Context Protocol (MCP) HTTP/SSE interface on port `8000`. We do not send commands over Kafka.
3. **Dual Clocks:**
4. **Simulation Time (`ts`):** RFC 3339 UTC string with second precision ending in `'Z'` (e.g. `2026-10-18T14:00:00Z`). All sensor timestamps reflect simulation time.
5. **Wall-Clock Time:** Used only inside the Twin process to pace the 10 Hz loop (sleeping ~100 ms between ticks).
6. **Sub-300-Byte Payload Budget:** Every JSON payload pushed to Kafka stays strictly <= 300 bytes.
7. **Cryptographic Signatures:** Every reading is signed by the device's private Ed25519 key. The signature is placed in the `sig` field of the Kafka envelope.
8. **Ground Truth Isolation:** The local SQLite database (`ground_truth.db`) records true physical values and injected attacks. Neither City Brain nor Guardian can query this file directly. They must rely on public MCP endpoints and Kafka messages.

---

2. Quickstart: Running the System

Option A: Docker Compose (Recommended for Teams)

This launches official Apache Kafka (`apache/kafka:latest` in KRaft mode), the Twin Layer daemon (simulation + MCP server), and the Streamlit 3D Dashboard.

```
# 1. Start all services
docker compose up -d

# 2. Check container status
docker compose ps

# 3. View Twin Layer logs
docker compose logs -f twin_layer
```

Network Endpoints

Service	Host Address	Docker Internal Address	Description
Apache Kafka	`localhost:19092`	`kafka:9092`	Outbound telemetry stream (`twin.sensors`)
MCP Command Server	`http://localhost:8000/sse`	`http://twin_layer:8000/sse`	JSON-RPC actuation and query endpoints
Red Team Adversary API	`http://localhost:8100/api/...`	`http://twin_layer:8100/api/..`	Isolated control endpoints (attacks, status; requires X-Red-Team-Token)
3D Operations Console	`http://localhost:8501`	`http://dashboard:8501`	Streamlit tactical dashboard

Option B: Local Python Development (No Docker)

If you want to run or debug the code directly in your local Python environment:

```
# 1. Clone repository & create virtual environment
python3 -m venv venv
source venv/bin/activate

# 2. Install dependencies
pip install -r requirements.txt

# 3. Run the full test suite (41 tests)
pytest tests/ -v

# 4. Run the Twin Layer runtime
# Note: If no Kafka broker is reachable, it automatically buffers to an internal local sink.
python -m twin.main

# 5. In a second terminal, launch the Streamlit console
streamlit run dashboard.py
```

---

3. Partner Integration: Layer 2 (City Brain)

City Brain consumes sensor telemetry from Kafka to detect city incidents, then issues actuation commands through MCP to resolve them.

Step 1: Consuming Sensor Telemetry

Connect any standard Kafka client to `localhost:19092` and subscribe to `twin.sensors`.

```
import json
from confluent_kafka import Consumer

conf = {
    'bootstrap.servers': 'localhost:19092',
    'group.id': 'city-brain-worker',
    'auto.offset.reset': 'latest'
}
consumer = Consumer(conf)
consumer.subscribe(['twin.sensors'])

while True:
    msg = consumer.poll(timeout=1.0)
    if msg is None:
        continue
    if msg.error():
        print(f"Consumer error: {msg.error()}")
        continue

    payload = json.loads(msg.value().decode('utf-8'))
    device_id = payload['device_id']
    reading_value = payload['reading']
    reading_type = payload['type']
    timestamp = payload['ts']
    print(f"[{timestamp}] {device_id} ({reading_type}): {reading_value}")
```

Step 2: Calling MCP Actuation Tools

Use the standard HTTP POST interface or the official Python MCP SDK to trigger city actions.

Python MCP Client Example

```
import asyncio
from mcp.client.session import ClientSession
from mcp.client.sse import sse_client

async def trigger_emergency_dispatch():
    async with sse_client("http://localhost:8000/sse") as (read_stream, write_stream):
        async with ClientSession(read_stream, write_stream) as session:
            await session.initialize()

            # 1. Dispatch emergency units to Zone 1
            dispatch_result = await session.call_tool(
                "dispatch",
                arguments={"incident_type": "fire", "zone_id": "zone-1"}
            )
            print("Dispatch Response:", dispatch_result.content[0].text)

            # 2. Adjust traffic lights to green wave along Avenue Habib Bourguiba
            signal_result = await session.call_tool(
                "set_signal_plan",
                arguments={"device_id": "z1-tc1", "target_state": "green_wave"}
            )
            print("Traffic Signal Response:", signal_result.content[0].text)

            # 3. Open flood drainage valves during high water levels
            valve_result = await session.call_tool(
                "set_valve",
                arguments={"device_id": "z1-wl1", "target_state": "open"}
            )
            print("Valve Response:", valve_result.content[0].text)

asyncio.run(trigger_emergency_dispatch())
```

4. Partner Integration: Layer 3 (Guardian)

Guardian verifies the cryptographic integrity of every Kafka reading, detects cyberattacks, and enforces containment.

Step 1: Verifying Ed25519 Signatures

Each device has an Ed25519 public key. Fetch it once via MCP `get_device_key(device_id)`, then cache it. Verify each message against the canonical JSON string.

```
import json
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PublicKey

def canonical_bytes(payload: dict) -> bytes:
    """Reconstruct exact canonical JSON bytes excluding the signature."""
    ordered_payload = {
        "reading_id": payload["reading_id"],
        "device_id": payload["device_id"],
        "type": payload["type"],
        "ts": payload["ts"],
        "reading": payload["reading"],
        "status": payload["status"]
    }
    return json.dumps(ordered_payload, separators=(',', ':'), sort_keys=False).encode('utf-8')

def verify_telemetry(payload: dict, public_key_hex: str) -> bool:
    sig_bytes = bytes.fromhex(payload['sig'])
    pub_key_bytes = bytes.fromhex(public_key_hex)
    public_key = Ed25519PublicKey.from_public_bytes(pub_key_bytes)

    try:
        public_key.verify(sig_bytes, canonical_bytes(payload))
        return True
    except Exception:
        return False
```

Step 2: Quarantining Compromised Devices

When a signature fails or sensor readings drift into physically impossible values, invoke MCP containment tools:

```
async def quarantine_rogue_sensor(session, device_id: str, reading_id: str):
    # 1. Sever network authority (drops subsequent readings from stream)
    await session.call_tool("quarantine_device", arguments={"device_id": device_id})

    # 2. Mark falsified reading in audit ledger
    await session.call_tool("rollback_reading", arguments={"reading_id": reading_id})

    # 3. Isolate physical zone circuit
    await session.call_tool("isolate_device", arguments={"device_id": device_id})
```

5. Complete MCP Tool Reference

All tools are accessible via `http://localhost:8000/sse` using standard MCP JSON-RPC format.

Queries

Tool	Parameters	Returns	Description
`list_devices`	*(none)*	`list[dict]`	Returns all 42 registered sensors, types, zones, and status.
`get_reading`	`device_id` (str)	`dict`	Returns the latest broadcast reading for a specific device.
`get_readings`	`device_id` (str), `limit` (int), `since_ts` (str, optional)	`list[dict]`	Returns chronological historical readings with optional RFC 3339 filter.
`get_device_state`	`device_id` (str)	`dict`	Returns current operational status, isolation state, and active actuation.
`get_city_state`	*(none)*	`dict`	Returns zone-by-zone aggregate metrics across Greater Tunis.
`get_device_key`	`device_id` (str)	`dict`	Returns Ed25519 public key hex string and key status (`active` / `revoked`).

Actuations (City Brain Controls)

Tool	Parameters	Allowed Target States	Description
`dispatch`	`incident_type` (str), `zone_id` (str)	`medical`, `fire`, `police`, `hazmat`	Dispatches emergency units; reduces active incident count.
`set_signal_plan`	`device_id` (str), `target_state` (str)	`green_wave`, `all_red`, `adaptive`	Controls traffic camera intersections; reduces traffic congestion.
`set_valve`	`device_id` (str), `target_state` (str)	`open`, `half`, `closed`	Controls flood drainage valves; lowers flood water level.
`switch_grid`	`device_id` (str), `target_state` (str)	`grid_a`, `grid_b`, `island`	Reroutes power substation feeder lines.
`get_command_result`	`command_id` (str)	-	Polls execution status of previously submitted command.

Containment (Guardian Controls)

Tool	Parameters	Description
`quarantine_device`	`device_id` (str)	Revokes device network authority; stops broadcasting its packets to Kafka.
`release_device`	`device_id` (str)	Restores network authority after threat removal.
`isolate_device`	`device_id` (str)	Disconnects device's physical subsystem.
`reinstate_device`	`device_id` (str)	Reconnects physical subsystem.
`rollback_reading`	`reading_id` (str)	Flags a specific falsified reading as `rolled_back` in the historical buffer.

6. Testing & Benchmark Reproduction

Running Automated Test Suites

Our test suite validates physics kinetics, cryptographic signing, adversary injection, sub-300-byte budget compliance, MCP tool calls, and dashboard DOM mounting:

```
# Run all 41 unit and conformance tests
pytest tests/ -v
```

Running the 30-Second Stress Benchmark

This benchmark tests our streaming producer under continuous 10 Hz load across all 42 devices:

```
pytest tests/test_qa_conformance.py -k test_30_second_high_throughput_stress_benchmark -s
```

Expected Benchmark Results:
- Cycles executed: 300 cycles (30 simulated seconds).
- Messages produced: 12,600 payloads.
- Throughput rate: >2,500 messages/sec (surpasses the 420 msgs/s requirement by 6x).
- Memory delta: <30 MB RSS delta over entire run.

Inspecting the Ground Truth Ledger Directly

To inspect the ground-truth SQLite database locally:

```
sqlite3 ground_truth.db "SELECT count(*) FROM readings_truth;"
sqlite3 ground_truth.db "SELECT * FROM attacks_injected WHERE active = 1;"
sqlite3 ground_truth.db "SELECT device_id, type, zone_id, status FROM devices LIMIT 10;"
```

7. Troubleshooting & Gotchas

1. `ConnectionRefusedError: [Errno 61]` on Kafka port 19092

- **Cause:** Apache Kafka in KRaft mode takes 5 to 10 seconds to finish controller quorum elections on initial boot.
- **Fix:** Wait 10 seconds or check container health with `docker compose ps`. Our `twin.main` runner automatically retries connecting up to 10 times.

2. `sqlite3.OperationalError: database is locked`

- **Cause:** Concurrent reads from the Streamlit dashboard while the simulation engine is inserting batches at 10 Hz.
- **Fix:** We resolved this in `twin/db_ledger.py` by enabling Write-Ahead Logging (`PRAGMA journal_mode = WAL;`) and a 10-second busy timeout (`PRAGMA busy_timeout = 10000;`). If you open the database in a custom script, always use `timeout=10.0`.

3. Port Conflicts (`8000` , `8501` , `19092`)

- If port `8000` is already in use by another local web service, override it via environment variable:
  `bash MCP_PORT=8001 python -m twin.main`
- If port `8501` is taken, launch Streamlit on another port:
  `bash streamlit run dashboard.py --server.port 8502`